

PHASE 6

FSD Documentation Format – Project Documentation

Project Title	Script-Controlled ACL – Restrict Record Access Based on Field Value
Team ID	SWTID-2026-1695
Project Name	Skill Wallet

1. Introduction

Project Title: Script-Controlled ACL – Restrict Record Access Based on Field Value

Team Members: [Enter Team Member Names and Roles]

2. Project Overview

Purpose

To restrict Read, Create, Write, and Delete access to Skill Wallet Request records in ServiceNow dynamically, based on the value of the record's own Department field, rather than relying only on the user's assigned role.

Features

- Custom roles: u_swt_admin (full access) and u_swt_user (department-restricted access)
- Custom table: Skill Wallet Request (u_skill_wallet_request) with Employee, Department, Skill Name, Wallet Points, and Status fields
- Four Script-Controlled ACLs — Read, Create, Write, and Delete — each checking the Department field value against the logged-in user
- Delete restricted exclusively to the u_swt_admin role

3. Architecture

- Frontend (equivalent): ServiceNow native list and form views (Now Platform UI) — no custom frontend framework (React, etc.) was built; the platform's own UI renders the Skill Wallet Request table
- Backend (equivalent): Script-Controlled ACL condition scripts (server-side GlideScript) evaluated by the ServiceNow ACL engine, in place of a Node.js/Express backend
- Database: ServiceNow platform database — custom table u_skill_wallet_request and the platform sys_user / sys_user_role tables, in place of an external MongoDB/SQL database

4. Setup Instructions

Prerequisites

- A ServiceNow Personal Developer Instance (PDI) with admin access (in place of Node.js/MongoDB dependencies)

Installation / Configuration Steps

- Create roles u_swt_admin and u_swt_user
- Create test users and assign roles and department values
- Create the Skill Wallet Request table and its fields
- Create the four Script-Controlled ACL records (Read, Create, Write, Delete) as documented in Phase 5

5. Folder Structure (Application Structure)

Since this project has no client/server folder structure (it is configured entirely within ServiceNow), the equivalent structure is:

- Roles: u_swt_admin, u_swt_user

- Table: u_skill_wallet_request (5 custom fields)
- Access Control records: 4 (Read, Create, Write, Delete) on u_skill_wallet_request

6. Running the Application

- Log in to the ServiceNow instance (in place of running npm start for frontend/backend)
- Use Impersonate User to switch between Alice Admin, Ben HR, and Carol IT
- Navigate to the Skill Wallet Request table (list/form view) to observe the access behaviour enforced by each ACL

7. API Documentation (ACL Script Documentation)

In place of REST API endpoints, this project exposes its logic as ACL condition scripts. Read / Create / Write ACL script (identical logic applied per operation):

```
answer = false;
if (gs.hasRole('u_swt_admin')) {
    answer = true;
} else {
    var gr = new GlideRecord('sys_user');
    if (gr.get(gs.getUserID())) {
        if (current.u_department == gr.u_department) {
            answer = true;
        }
    }
}
}
```

Delete ACL script:

```
answer = false;
if (gs.hasRole('u_swt_admin')) {
    answer = true;
}
```

8. Authentication

Authentication is handled natively by the ServiceNow platform login (in place of token/session-based authentication). Authorization is layered: first the user must hold a valid session; then, for every Read, Create, Write, or Delete on the Skill Wallet Request table, the matching Script-Controlled ACL evaluates the user's role (u_swt_admin) and, if absent, compares the record's Department field to the user's own department field before granting or denying the action.

9. User Interface

[SCREENSHOT PLACEHOLDER: Screenshot(s) of the Skill Wallet Request list view and form view]

10. Testing

Testing strategy and tools are documented fully in Phase 5 (Development & Testing Phase / UAT), including 12 UAT test cases covering Read, Create, Write, and Delete for both the admin role and department-restricted users, executed via the Impersonate User feature.

11. Screenshots or Demo

[SCREENSHOT PLACEHOLDER: Key screenshots from each milestone: Roles, Users, Table, all four ACL records, and at least two Access Granted / Access Denied outcomes]

12. Known Issues

[Enter any known limitations observed during testing, e.g., ACL relies on the u_department field being kept consistent between sys_user and u_skill_wallet_request records.]

13. Future Enhancements

- Extend the same script-controlled ACL pattern to additional tables or fields (e.g., restrict by Location or Project)
- Add a UI Policy to visually hide the Department field for non-admin users, as a usability improvement layered on top of the existing ACL security
- Add reporting/audit logging of denied access attempts for compliance visibility